Super Simple Software Factory

Installation guide for Linux, macOS and Windows

SSSF is installed in two moves: copy the `sssf` skill into the repository you want the factory to work on, then run its installer, which stamps the ADW scripts, the agent roster, prompts and recipes into that repository. Everything below is run from the **root of that target repository**, because that is where the installer writes.

1. Prerequisites
2. Copy the skill into your repository
3. Stamp the factory
4. Set your API keys
5. Make it a git repository
6. Wire your real test command
7. Smoke test
8. Windows notes
9. Updating an installation
10. Troubleshooting

1 Prerequisites

Tool	Needed for	Linux / macOS	Windows
Git	Required. Commit phases, change capture, the write boundary.	Your package manager (`apt install git` , `brew install git`)	**Git for Windows**, at its default location: `winget install --id Git.Git -e`
uv	Required. Runs every ADW and the installer, and fetches Python and the dependencies for you.	`curl -LsSf https://astral.sh/uv/install.sh | sh`	`powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"`
pi	The default coding agent (`coding_agent: pi`).	Install from `github.com/mariozechner/pi-coding-agent` , then check with `pi --version` . Not needed if every agent uses Claude Code.	
Claude Code	Only for agents set to `coding_agent: claude_code` .	`claude` on PATH, logged in, or `ANTHROPIC_API_KEY` set.	
just	Optional. The stamped recipes (`just demo` , `just obs` , …).	`brew install just` or `uv tool install rust-just`	`winget install --id Casey.Just --exact`
sqlite3	Optional. The trace-reading recipes (`just sessions` , `just phases`).	Usually preinstalled; else your package manager	Command-line tools from `sqlite.org/download.html` , with `sqlite3.exe` on PATH
bun	Optional. The trace visualizer (`just obs`).	`curl -fsSL https://bun.sh/install | bash`	`powershell -c "irm bun.sh/install.ps1 | iex"`

You also need an API key for every model provider your agent roster names (step 4).

2 Copy the skill into your repository

LINUX / MACOS / GIT BASH

```
mkdir -p .claude/skills
cp -r /path/to/ssaf/.claude/skills/sssf
.claude/skills/
```

WINDOWS POWERSHELL

```
New-Item -ItemType Directory -Force
.claude\skills
Copy-Item -Recurse
C:\path\to\ssaf\.claude\skills\sssf
.claude\skills\
```

> **Using Claude Code?** Once the skill is in place you can type `/sssf install` inside Claude Code and the agent performs steps 3 to 7 for you. The manual steps below are what it does.

A skill installed for your user instead (`~/.claude/skills/sssf`) works too; use that path in the next command.

3 Stamp the factory

```
uv run .claude/skills/sssf/scripts/install.py
```

The same command works in every shell. It creates:

Path	What it is
`adws/adw_*.py`	The starter workflows (ADWs), e.g. `adw_scout` , `adw_plan_build_test` , `adw_simple_sdlc`
`adws/adw_modules/`	The factory's own code. Protected: agents may not edit it.
`adws/adw_sssf_config/sssf.config.yaml`	The agent roster: models, tools, write permissions, timeouts
`adws/adw_data/prompt_engineering/`	Each agent's prompts. Yours to edit.
`adws/adw_data/harness_engineering/`	Pi extensions and the Context7 MCP config
`justfile` , `.env.sample`	Recipes and the environment template
`.gitignore` entries	Runtime data (`adws/adw_data/sessions/` , `sssf.db`), `.env` , bytecode

Re-running it is safe: existing files are skipped and listed, so a second run doubles as a drift check.

4 Set your API keys

LINUX / MACOS / GIT BASH

```
cp .env.sample .env
```

WINDOWS POWERSHELL

```
Copy-Item .env.sample .env
```

Then fill in `.env`. Which keys you need depends on the `model:` lines in `sssf.config.yaml`: each is written `provider/model-id`, and the provider decides the key. The starter roster deliberately mixes three providers:

Starter model	Used by	Key
`google/gemini-3.6-flash` (via OpenRouter)	default, builder, scout	`OPENROUTER_API_KEY`
`fireworks/…/kimi-k3`	planner	`FIREWORKS_API_KEY`
`openai/gpt-5.6-terra`, `openai/gpt-5.6-luna`	reviewer, documenter	`OPENAI_API_KEY`

> **One key instead of three:** delete the per-agent `model:` lines so every agent inherits `defaults.model`. A missing key is not caught at startup; it fails when that agent runs.

Context7 (library docs) needs no key; `CONTEXT7_API_KEY` only raises its rate limit. The docs phase sends library names, versions and the first 300 characters of your request to Context7. Set `context7.send_request: false` to send names only, or `context7.enabled: false` to turn it off.

5 Make it a git repository

Workflows that commit need a repository with at least one commit. If the target is not one yet:

```
git init
git add -A
git commit -m "Install sssf"
```

Start runs from a clean tree. A commit phase only commits what the run changed and leaves your own uncommitted files out, but a clean tree avoids the question.

6 Wire your real test command

> **Do this before trusting any test workflow.** `adws/adw_modules/quality.py` ships with placeholder commands. On Linux and macOS they print a notice and pass, so a fresh install reports a green test phase that tested nothing.

Open `adws/adw_modules/quality.py` and replace each `_placeholder(...)` with the real command, as a list:

```
argv=["uv", "run", "pytest", "-q"]      # or ["bun", "test"], ["npm", "run", "lint"], …
```

Delete the blocks you don't need and remove them from `run_quality()`.

7 Smoke test

```
just demo        # two cheap, read-only runs, end to end
just sessions    # what just happened
just obs         # the trace UI at http://localhost:4601 (needs bun)
```

Without `just`, run the first half of the demo directly:

```
uv run adws/adw_prompt.py "reply with a one-line summary of this repo" --agent scout
```

Success means the config validated, an agent ran, its report parsed, and the trace landed in `adws/adw_data/sssf.db`. Fix any failure here before running larger workflows: every multi-agent workflow rides this same path. The trace UI needs that database, so run the demo before `just obs`.

8 Windows notes

The justfile runs on Git for Windows' shell

The stamped `justfile` starts with:

```
set windows-shell := ["C:/Program Files/Git/bin/sh.exe", "-cu"]
```

On Windows, `just` otherwise runs recipes with PowerShell or `cmd`, which do not understand the POSIX syntax the recipes use. This line makes every recipe run in Git for Windows' `sh`, so the recipes behave the same on every OS. Linux and macOS ignore it.

- **Git installed somewhere else** (for example per-user under `%LOCALAPPDATA%\Programs\Git`)? Edit the path in that line. Find it with `where git`: the shell is `bin\sh.exe` next to the `cmd` folder that output names.
- Use forward slashes in the path, as shown, or doubled backslashes.

Running the commands

- `uv run …`, `git …` and `just …` commands work unchanged in PowerShell.
- Windows PowerShell 5 has no `&&`; run chained commands one per line, as this guide does.
- All files and agent output are read and written as UTF-8 regardless of the Windows code page, so non-English text and emoji in prompts or fetched docs are safe.

Behaviour that differs on Windows

- **Stopping a run.** On Linux and macOS, killing an ADW (or an agent timing out) also kills every process the agent started. On Windows only the agent process itself is terminated; a test server it launched may need closing by hand.
- **Placeholder test commands** may fail with exit 127 instead of passing, because `echo` is not a program on Windows unless Git's Unix tools are on PATH. Either way, step 6 replaces them.

9 Updating an installation

```
uv run .claude/skills/sssf/scripts/install.py --force
```

Copy the new skill version into `.claude/skills/sssf` first. `--force` refreshes the stamped code, but it also overwrites **all** stamped files, including `sssf.config.yaml`, your prompts and `quality.py`. **Commit first**, then re-apply your edits from the diff. The one file it never overwrites is a `justfile` that did not come from sssf; it reports that file as kept.

10 Troubleshooting

Symptom	Cause	Fix
`config validation failed` before anything runs	A model name is ambiguous or unknown, or a prompt file is missing	Write models as `provider/model-id`; check `pi --list-models`
An agent fails partway through a workflow	Its provider's API key is not set	Add the key to `.env` (step 4)
A commit phase says “not a git repository” or “nothing to commit”	No repository, or the run changed nothing	Step 5
Test phase is green but nothing was tested	Placeholder commands in `quality.py`	Step 6
`just obs` says `sssf.db not found`	No run has happened yet	Run `just demo` first
On Windows, `just` cannot find `sh.exe`	Git for Windows is missing or not at the default path	Install it, or edit the `windows-shell` line (step 8)
A run seems stuck	An agent stopped responding	`just procs <adw_id>` shows its processes; each agent call is also killed after `timeout_s` (default one hour, set in `sssf.config.yaml`)
The docs phase reports errors	Context7 unreachable or rate-limited; the run continues without docs	Allow `mcp.context7.com` on your network, or set `CONTEXT7_API_KEY`
An agent's change was rolled back and the phase failed	It wrote outside its `writes:` permission	Expected. Widen that agent's `writes:` if the change was legitimate

More detail: the repository README and `.claude/skills/sssf/references/config.md`.